



1. (15 نمره)

الف - (5 نمره) شبه کدی بنویسید که دو رشته حرفی را با هم مقایسه کند، در صورتی که آن دو رشته برابر بودند مقدار 1 و در غیر این صورت 0 را چاپ کند (راهنمایی: در نوشتن شبه کد فرض کنید تابع `len()` را برای به دست آوردن طول یک رشته در اختیار دارید).

ب - (10 نمره) دو تابع زیر را کامل کنید. تابع `getLength` باید طول یک رشته را محاسبه و برگرداند و سپس با استفاده از این تابع، تابع `compareStrings` را طوری کامل کنید که دو رشته را مقایسه کند؛ در صورتی که این دو رشته برابر بودند مقدار 1 و در غیر این صورت مقدار 0 را برگرداند.

```
int getLength(char* str) {  
    ...  
}
```

```
int compareStrings(char* str1, char* str2){  
    ...  
}
```

2. (20 نمره) فری کثیف صاحب یک ساندویچی در مرکز شهر است ولی متاسفانه مشتری زیادی ندارد، چون سطح بهداشت ساندویچ های فری کثیف بسیار بالاتر از استاندارد های بین المللی است. او برای راهکار جذب مشتری تصمیم میگیرد برای هر ساندویچ خود، یک تخفیف بگذارد تا تعداد مشتری ها افزایش یابد.

الف - (5 نمره) یک `struct` به نام `Sandwich` با فیلدهای زیر تعریف کنید:

- نام ساندویچ (رشته کاراکتری با طول نامشخص)
- قیمت ساندویچ (عدد صحیح)
- درصد تخفیف (عدد صحیح)

● ب- (5 نمره) تابع `searchSandwich` را با استفاده از توابع نوشته شده بالا طوری کامل کنید که نام یک ساندویچ به همراه آرایه منوی ساندویچی را بگیرد و به دنبال غذای مورد نظر در منو بگردد. در صورتی که این غذا وجود داشت، آدرس آن خانه از آرایه را برگرداند و در غیر این صورت، `NULL` برگرداند.

*** توجه داشته باشید که برای این سوال استفاده از هیچ کتابخانه ای به جز `<stdio.h>` مجاز نیست. در صورت نیاز به مقایسه رشته ها باید از توابعی که در سوال اول نوشتید استفاده کنید.

```
Sandwich* searchSandwich(char* name, Sandwich* dishes, int menu_size) {  
    ...  
}
```

● پ- (10 نمره) اصغر که متوجه تخفیف های ساندویچی فری کثیف شده، تصمیم میگیرد تا ناهار را در این ساندویچی بخورد اما نمیداند که پول آن برای خرید یک ساندویچ مشخص کافی است یا نه. با کامل کردن تابع `canAfford` به اصغر کمک کنید تا بفهمد آیا با مقدار پولی که دارد، میتواند ساندویچ دلخواهش را بخرد یا نه. اول باید با استفاده از تابع قسمت الف بررسی کنید که آیا در منو، ساندویچ مدنظر اصغر وجود دارد یا نه. اگر وجود داشت بررسی کنید اگر پول اصغر کافی باشد، باید نام، قیمت اصلی و قیمت نهایی (بعد از اعمال تخفیف) که اصغر باید پرداخت کند چاپ شود. اگر پول کافی نبود یا ساندویچ وجود نداشت نیز پیام های مناسب طبق مثال ها چاپ شود.

```
void canAfford(int money, char* name, Sandwich* dishes, int menu_size){  
    ...  
}
```

مثال

- اگر ساندویچ در منو وجود نداشت:

Sandwich not found

- اگر پول برای خرید ساندویچ کافی نباشد:

Money is not enough

- اگر پول به اندازه ساندویچ خواسته شده باشد: (اطلاعات غذا مطابق نمونه چاپ شوند)

Sandwich: bandari

Price: 170

Discounted Price: 144.500000

3. (15 نمره) می‌خواهیم خانه‌های یک شهر را پلاک‌گذاری کنیم طوری که هیچ دو خانه‌ای پلاک یکسان نداشته باشند.

برای این کار، یک "دفتر ثبت پلاک‌ها" داریم که حداکثر می‌تواند اطلاعات MAX خانه را نگه دارد.

```
#define MAX 100
```

```
...
```

الف - (2.5 نمره) یک struct به نام House تعریف کنید که بیانگر یک خانه با فیلدهای زیر است:

- شماره پلاک (عدد صحیح)
- نام خیابان (رشته کاراکتری با حداکثر طول 50 کاراکتر)
- شماره واحد (عدد صحیح)

ب - (2.5 نمره) یک struct دیگر به نام Registry تعریف کنید شامل:

- آرایه‌ای از House با اندازه MAX
- تعداد خانه‌های ثبت‌شده تا این لحظه

پ - (10 نمره) تابع زیر را طوری کامل کنید که اشاره گر به یک رجیستری و اطلاعات یک خانه جدید را بگیرد. اگر پلاک قبلاً ثبت نشده بود، خانه را به انتهای آرایه رجیستری اضافه کند و مقدار 1 برگرداند. اگر پلاک تکراری بود، هیچ چیزی اضافه نکند و مقدار 0 برگرداند؛ و اگر رجیستری پر بود، مقدار 1- را برگرداند.

راهنمایی: تابع strcpy یک تابع در کتابخانه <string.h> است. این تابع دو رشته را به عنوان آرگومان می‌گیرد و رشته دوم را در رشته اول کپی می‌کند. در صورت نیاز می‌توانید از این تابع استفاده کنید.

```
int registerHouse(Registry* reg, int plate, char street[], int unit){  
    ...  
}
```



4. ****امتیازی (20 نمره) صف (Queue)** در برنامه نویسی یک ساختمان داده خطی است که

داده‌ها را به ترتیب ورود و خروجشان مدیریت می‌کند. این ترتیب بر اساس اصل **اولین ورودی، اولین خروجی (FIFO)** تعیین می‌شود؛ مانند صف نانویی که اولین فردی که وارد صف می‌شود، اولین فردی است که نوبتش می‌رسد و خارج می‌گردد (به جز احتمالاً در ایران 😊). این ساختمان داده برای مدیریت وظایف، پردازش درخواست‌ها و ارتباطات سیستم‌ها کاربرد فراوان دارد.

یک سیستم پشتیبانی، پیام‌های کاربران را به ترتیب ورود نگه می‌دارد تا اپراتور به ترتیب به آن‌ها رسیدگی کند. شما باید یک صف ساده بسازید که عملیات اصلی صف را پشتیبانی کند.

ویژگی‌های صف:

- هر پیام شامل: شناسه پیام، نام فرستنده و متن پیام است.
- صف ظرفیت ثابت CAP دارد.

#define CAP 10

- صف با **آرایه‌ی خطی ساده** پیاده‌سازی می‌شود. بنابراین هنگام حذف یک عنصر از ابتدای صف (عملیات حذف همواره از ابتدای صف اتفاق می‌افتد)، باید عناصر باقی‌مانده یک خانه به سمت چپ جابه‌جا شوند تا ترتیب حفظ شود.

الف- (2.5 نمره) یک struct به نام Message با فیلدهای زیر تعریف کنید که بیانگر یک پیام است.

- شناسه (id) به صورت عدد صحیح
- نام فرستنده به صورت رشته (حداکثر 20 کاراکتر)
- متن پیام به صورت رشته (حداکثر 100 کاراکتر)

ب- (2.5 نمره) یک struct به نام Queue با فیلدهای زیر تعریف کنید که بیانگر صف پیام‌ها است:

- آرایه پیام‌ها
- اندیس محل درج بعدی
- تعداد عناصر فعلی صف

پ- (2.5 نمره) تابع زیر را طوری کامل کنید که یک صف تازه ساخته شده را آماده استفاده کند.

این تابع باید اندیس و شمارنده را به طور مناسب مقداردهی کند.

```
void initQueue(Queue* q) {  
    ...  
}
```

ث- (2.5 نمره) دو تابع زیر را طوری کامل کنید که خالی بودن یا پر بودن صف را بررسی کنند.

```
int isEmpty(Queue* q) {  
    ...  
}  
  
int isFull(Queue* q) {  
    ...  
}
```

ج- (5 نمره) تابع زیر را طوری کامل کنید که یک پیام جدید را در جای مناسب در صف اضافه کند. در صورت درج موفق، تابع باید مقدار 1 را برگرداند؛ اگر صف پر بود، تابع باید مقدار 0 را برگرداند.

```
int enqueue(Queue* q, Message msg) {  
    ...  
}
```

چ- (5 نمره) تابع زیر را طوری کامل کنید که پیام اول صف حذف کرده و آن را برگرداند. در صورتی که صف خالی باشد، تابع باید Null برگرداند.

```
Message* dequeue(Queue* q) {  
    ...  
}
```

⚠️ در تحویل تمرین، حتما به نکات زیر توجه نمایید:

❖ هیچ گونه شباهتی در انجام تمرین بین افراد مختلف پذیرفته نمی‌شود. در صورت کشف هرگونه تقلب مطابق قوانین درس با افراد خاطی برخورد خواهد شد.
○ پیشنهاد می‌شود از امکان رفع اشکال خصوصی، حداکثر استفاده را نمایید!

❖ پاسخ هر پرسشی که امکان آپلود برای آنها فراهم شده است را به صورت یک فایل pdf. و با نام `HW8_<StdID>` آپلود نمایید. (که StdID شماره دانشجویی شماست. به عنوان مثال اگر شماره دانشجویی کسی 810100000 باشد، فایل بارگذاری شده او باید به نام HW8_810100000 باشد)

❖ مهلت تحویل تمرین تا 1404/10/18 می‌باشد.
○ تمرین تنها در سامانه Elearn دریافت خواهد شد و از ایمیل آن خودداری نمایید.
○ مهلت تحویل تمرین، تمدید نخواهد شد؛ در منظم بودن تحویل، کوشا باشید.

❖ پس از مطالعه کامل تمرین، در صورت هرگونه ابهام با [امین غلامی](#) یا [آرش قربانخانی](#) در ارتباط باشید.
○ موضوع ایمیل را `HW8 ICSP` وارد نمایید.